

Elementopia System Architecture & Implementation Mapping

Technical Reference Document & Module Blueprint

1. Hybrid Persistence Layer Analysis

Elementopia implements a resilient, hybrid sync-and-cache framework that bridges Supabase cloud persistence with local-first graceful degradation.

Cloud Mode (Preferred)

- **Onboarding:** Nicknames entered via the NicknameGate trigger auto-registration in the PostgreSQL database (UserController.java → /api/users/login) with frictionless STUDENT user provisioning.
- **Security:** Employs temporary stateless session nicknames that act as keys for JWT creation.
- **Retrieval:** Synchronizes data via GET /api/users/{username} to keep the client aligned with the cloud state.

Offline Mode (Local Fallback)

- **Local Profiles:** Stores fallback accounts in the browser's localStorage under elementopia_users if backend APIs are down.
- **Graceful Degradation:** Allows active gameplay session saves and profile edits offline, resolving the SDD SessionDataDispatcher constraint to handle network anomalies without crashing.

Data Reconciliation Gap Warning

Currently, there is **no offline-to-cloud batch reconciliation or merge logic** in UserService.jsx. Profiles created or updated offline remain local to that specific browser instance and will not sync back to Supabase automatically when an internet connection is re-established. Implementing an auto-sync check on network status changes is highly recommended for future sprints.

2. Specification-to-Code Mapping (Modules 1.1 – 1.5)

The following matrix serves as a blueprint mapping the core specification modules to their corresponding frontend React files and backend Spring Boot controllers/services:

Architecture Matrix

Module	Spec Description	Frontend Component(s)	Backend Java Feature File(s)	Key Interactions / Roles
1.1	Resonance Domain Interaction	ElementopiaGame.jsx GameBoard.jsx ElementTile.jsx	ReactionController.java ValidationService.java TelemetryService.java	Handles drag-and-drop elements on a workspace canvas workbench, evaluating combinations.
1.2	Byproduct Feedback	GameBoard.jsx (Byproduct Alert Overlay)	FeedbackLibraryService.java FailedAttemptLog.java	Intercepts synthesis failures to display micro-lessons explaining byproduct formulas rather than "Wrong" text.
1.3	Hazmat Failsafe	GameBoard.jsx ElementTile.jsx (Disabled visual)	FailsafeMonitorController.java SlidingWindowEvaluationService.java FailsafeTelemetryService.java	Monitors failures; if 5 failures in 15 seconds occur, it visually grays out non-solution path tiles to stop random guessing.
1.4	Learning Progression	DashboardHub.jsx StoryCard.jsx	ProgressionController.java ProgressionValidationService.java RoomCompletionState.java	Tracks locked and unlocked cavern domains sequentially based on prerequisite completion.
1.5	Random Generation	GameBoard.jsx (Challenge setup)	ChallengeController.java ChallengeRandomizerService.java ChallengeRepository.java	Algorithmic reshuffling/shuffling of target compounds per level session to limit rote repetition.

3. Recommendations for Future Development

- **Implement Sync-Reconciliation Hook:** Incorporate a sync manager in UserService.jsx that detects network online events (via browser navigator.onLine listeners) and automatically pushes locally queued performance objects up to /api/users/register or logs endpoints.
- **Optimize Telemetry Log Latency:** Ensure that custom hooks monitoring state in TelemetryCaptureHook process batch updates before dispatching, guaranteeing writes stay under the SDD-mandated 2-second telemetry latency threshold.
- **Validate Failsafe Sliding Window in Memory:** Keep failsafe validation on the backend robust but secure, validating failure records with secure timestamps to prevent bypass injection by cheating clients.